

# CONTENTS

|                                       |       |
|---------------------------------------|-------|
| Document Intelligence — Documentation | ..... |
| Documentation map                     | ..... |
| Business                              | ..... |
| Architecture                          | ..... |
| Overview & design                     | ..... |
| Flows (architecture/flows/)           | ..... |
| Technical / Dev                       | ..... |
| Conventions across these docs         | ..... |

# Document Intelligence — Documentation

**Status:** Living index · **Last updated:** 2026-06-24

This is the documentation landing page for the **Document Intelligence** platform — the service that turns a bank client's KYC documents (PDF, DOCX, JPEG, PNG, plain text) into a **versioned, per-client knowledge tree** and serves it to downstream services for search, single-document Q&A, and structured fact retrieval — **PII-safe throughout**.

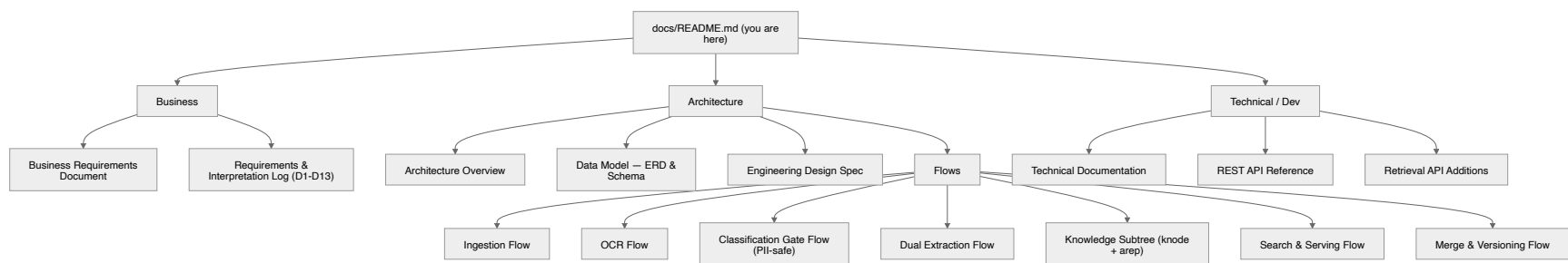
If you are new here, start with the audience-specific entry points:

- **Business / product** → [Business Requirements Document](#)
- **Architects / new engineers** → [Architecture Overview](#)
- **Developers / integrators** → [Technical Documentation](#) and the [API Reference](#)

Every document below is grounded in committed code under `di/` and traces back to the locked decisions (D1–D13) in the [requirements & interpretation log](#).

---

## Documentation map



## Business

For product owners, compliance stakeholders, and anyone asking *what the platform does and why*.

| Document                                                                        | What it covers                                                                                                                                                                          |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Business Requirements Document — Document Intelligence Platform</a> | The business need: PII-safe processing, the per-client knowledge tree, multi-tenant isolation, supported geographies/languages. Every requirement traces to a locked decision (D1–D13). |
| <a href="#">Requirements &amp; Interpretation Log (decisions D1–D13)</a>        | Append-only record of what the product owner asked for, how it was interpreted, and the design that followed. The source of truth for <i>why</i> each choice was made.                  |

## Architecture

For architects and engineers who need the system-level map: how a document travels from upload to knowledge tree, the data model behind it, and each stage in detail.

### OVERVIEW & DESIGN

| Document                                                                | What it covers                                                                                                                                                                                                    |
|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Document Intelligence — Architecture Overview</a>           | The system-level map: end-to-end document journey, FastAPI app structure, the two external systems (retrieval gateway, mock OCR), and the index-many / return-parent knowledge subtree at the center. Start here. |
| <a href="#">Data Model — Entity Relationship &amp; Schema Reference</a> | The Postgres 16 + pgvector + ltree schema: seven tables, HASH partitioning by <code>client_id</code> , RLS isolation, and the ERD. Grounded in <code>di/migrations/</code> and <code>di/store.py</code> .         |
| <a href="#">Engineering Design Spec</a>                                 | The <i>how it is built</i> companion to the BRD: design decisions, module layout, and data model rationale.                                                                                                       |

## FLOWS ( `ARCHITECTURE/FLOWS/` )

Each stage of the pipeline and the read side, documented against the code that implements it.

| Flow                                                                           | What it covers                                                                                                                                                            |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Ingestion Flow</a>                                                 | End-to-end: how one uploaded document becomes a versioned subtree, and the SSE stages the caller observes.                                                                |
| <a href="#">OCR Flow</a>                                                       | Turning raw bytes into an <code>OcrResult</code> (text + per-line geometry/confidence) via <code>extract_pages</code> — Azure Read v3.2 plus fallbacks.                   |
| <a href="#">Classification Gate Flow (PII-safe)</a>                            | The fail-safe, fully-offline chokepoint that produces the egress decision ( <code>SEND_TO_LLM</code> / <code>REDACT_THEN_SEND</code> / <code>DETERMINISTIC_ONLY</code> ). |
| <a href="#">Dual Extraction Flow</a>                                           | The deterministic path (always) and the LLM path (only when the gate clears egress), both yielding <code>ExtractedField</code> lists.                                     |
| <a href="#">The Knowledge Subtree — <code>knode</code> + <code>arep</code></a> | The two-table index-many / return-parent design and how the four properties (semantic, logical, contextual, accessibility) are realised.                                  |
| <a href="#">Search &amp; Serving Flow</a>                                      | The read side: hybrid scoped search, tree/fact/provenance endpoints, the capabilities manifest, and the access-aware masking projection.                                  |
| <a href="#">Cross-Document Merge &amp; Versioning Flow</a>                     | Confidence-weighted consolidation into <code>client_merged_fact</code> , plus copy-on-write versioning with content-hash dedup and the version delta feed.                |

## Technical / Dev

For engineers building, running, integrating with, or extending the platform.

| Document                                                                              | What it covers                                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Document Intelligence — Technical Documentation</a>                       | Reference for building, running, and integrating: the FastAPI app's three concerns (ingestion, storage, serving), delegated model access, graceful degradation, and optional dependencies.                     |
| <a href="#">Document Intelligence — REST API Reference</a>                            | The complete REST surface: ingestion (SSE), per-client tree traversal, hybrid search, provenance, capabilities, masked projections, and health probes. Tenant scoping and error envelope conventions included. |
| <a href="#">Retrieval Framework — API Additions Required by Document Intelligence</a> | Endpoints the external <code>retrieval</code> service must expose so this platform can delegate all model access (embed / LLM / rerank). Additive and backward-compatible; hand to the retrieval-repo team.    |

## Conventions across these docs

- **Tenancy:** every row, query, and API call is scoped to one `client_id` (supplied with the document; no entity resolution). RLS enforces isolation.
- **PII-safe by default:** the classification gate decides per document whether content may leave the local deterministic path. When in doubt, nothing leaves.
- **No model credentials live here:** embeddings, LLM completion, and reranking are delegated to the external `retrieval` service over HTTP; a deterministic stub keeps the pipeline runnable offline.
- **Grounded in code:** each doc cites the `di/` modules it describes. If a doc and the code disagree, the code wins — file a fix.